

INFORME CORPORATIVO: DISEÑO, IMPLEMENTACIÓN Y GESTIÓN DE BASES DE DATOS

PROYECTO INTERMODULAR ASIR 1 - TECHNOVA SOLUTIONS S.L.



Autor: Javier Ordóñez Muñoz

Módulo: Gestión de Bases de Datos

Ciclo: 1º Administración de Sistemas Informáticos en Red (ASIR)

Profesor: Jaime García Quevedo

ÍNDICE

1. INTRODUCCIÓN Y PROPÓSITO DEL PROYECTO INTERMODULAR

- 1.1. El Ecosistema TechNova: Integración de la Base de Datos en la Infraestructura
- 1.2. Objetivo del Proyecto: Gestión escalable de la información

2. ANÁLISIS DE LOS DATOS Y CICLO DE VIDA DE LA EMPRESA

- 2.1. El enfoque de negocio: Justificación operativa del dato
- 2.2. Pilar 1: Gestión del Talento y Recursos Humanos (RRHH)
- 2.3. Pilar 2: Infraestructura Física y Virtual
- 2.4. Pilar 3: Operaciones y Trazabilidad de Servicios
- 2.5. Pilar 4: Contabilidad y Facturación Escalable

3. DISEÑO LÓGICO Y ESTRUCTURAL

- 3.1. Fundamentos del Diseño Relacional
- 3.2. Etapa 1: Gestión de Identidades (RRHH)
- 3.3. Etapa 2: Control de Infraestructura y Virtualización
- 3.4. Etapa 3: Operaciones, Proyectos y Ciberseguridad
- 3.5. Etapa 4: Escalabilidad Financiera y Facturación
- 3.6. El Plano Lógico: Modelo Relacional y Normalización (3FN)

4. IMPLEMENTACIÓN ESTRUCTURAL (CÓDIGO DDL)

- 4.1. Traducción del Modelo Relacional a SQL (DDL)
- 4.2. Script de creación de la base de datos (Esqueleto de TechNova)

5. INSERCIÓN Y VIDA DEL DATO (CÓDIGO DML)

- 5.1. Población del Sistema: Vinculación Intermodular
- 5.2. Script de Inserción de Datos Operativos (DML)

6. EXPLOTACIÓN DE DATOS: GENERACIÓN DE INTELIGENCIA DE NEGOCIO

- 6.1. La importancia de la extracción de métricas para la gerencia
- 6.2. Auditoría de Recursos Humanos (Control de Carga Laboral)
- 6.3. Auditoría de Ciberseguridad (Trazabilidad y Mitigación de Incidentes)
- 6.4. Auditoría Contable (Automatización de Facturación)

7. ADMINISTRACIÓN, MANTENIMIENTO Y SEGURIDAD

- 7.1. Control de Acceso y Gestión de Privilegios (DCL)
- 7.2. Política de Copias de Seguridad y Disaster Recovery (Backups)

8. CONCLUSIÓN: ESCALABILIDAD Y VISIÓN GLOBAL DEL PROYECTO

- 8.1. Arquitectura preparada para el crecimiento corporativo
- 8.2. Reflexión Personal y Aprendizaje Adquirido

1. INTRODUCCIÓN Y PROPÓSITO DEL PROYECTO INTERMODULAR

1.1. El Ecosistema TechNova: Integración de la Base de Datos en la Infraestructura

Durante este primer año en el ciclo de ASIR, me he centrado en levantar la infraestructura tecnológica de TechNova Solutions S.L., una consultora especializada en ciberseguridad e inteligencia artificial. Hasta ahora, el proyecto se dividía por módulos técnicos: montar el hardware físico, instalar los sistemas operativos y planificar la red. Sin embargo, en un entorno real, tener servidores potentes y una red bien configurada no sirve de nada si no tenemos un control claro sobre la información que maneja la empresa.

Al plantear el día a día de la consultora, vi claramente que sin una base de datos central, la gestión sería imposible. Si un cliente nos llama, necesitamos saber al momento qué técnico lleva su proyecto, qué servidores estamos usando para su servicio y qué facturación tiene asociada. Por este motivo, el diseño de esta base de datos se ha convertido en el centro de la empresa. Su función es registrar y organizar cada dato para que toda la infraestructura física y lógica que he diseñado en las otras asignaturas tenga una utilidad real para el negocio.

1.2. Objetivo del Proyecto: Gestión escalable de la información

El objetivo de este documento es explicar cómo he diseñado la estructura de datos de TechNova desde cero. No me he limitado a crear tablas sueltas solo para cumplir con la nota, sino que he analizado los procesos internos reales de una consultora: desde que se contrata a un empleado o se hace el inventario de máquinas, hasta que se monta un servicio y se le cobra al cliente.

He planteado la base de datos pensando en el futuro de la empresa y su crecimiento. Si el día de mañana TechNova pasa de tener 20 empleados a 2.000, o de facturar diez proyectos a gestionar miles de contratos, la estructura tiene que aguantar todo ese volumen sin quedarse colgada ni mezclar datos.

Para lograr esto, he diseñado un modelo compuesto por 15 tablas. En las próximas páginas voy a detallar por qué he creado cada una de ellas, qué problemas del día a día resuelven y cómo se conectan entre sí. Mostraré cómo he configurado reglas y restricciones para asegurar que la información cuadre siempre, protegiendo al sistema frente a fallos humanos o borrados por accidente, y haciendo que sea mucho más fácil de administrar a largo plazo.

2. ANÁLISIS DE LOS DATOS Y CICLO DE VIDA DE LA EMPRESA

2.1. El enfoque de negocio: Justificación operativa del dato

Un error muy típico al diseñar bases de datos es empezar a crear tablas sin entender primero cómo funciona el negocio. En mi caso, he enfocado este proyecto con otra mentalidad: antes de escribir nada de código SQL, he analizado cómo se mueve la información dentro de TechNova. He organizado el funcionamiento de la empresa en cuatro pilares principales (Recursos Humanos, Infraestructura, Operaciones y Contabilidad), como se ve en el esquema adjunto. Siguiendo esta idea, cada registro que se guarda en el sistema tiene un motivo técnico o económico detrás; si un dato no sirve para saber qué ha pasado, no agiliza un proceso o no evita un fallo de seguridad, simplemente no lo incluyo en el modelo.

2.2. Pilar 1: Gestión del Talento y Recursos Humanos (RRHH)

Al ser una consultora tecnológica, lo más valioso de la empresa es el conocimiento de nuestros técnicos. La gestión de los empleados no podía quedarse en guardar nombres y correos; necesitaba reflejar los rangos y las capacidades reales del equipo. Para conseguirlo, he enlazado los departamentos directamente con el diseño de red, asignando las VLANs del módulo de Planificación de Redes, uniendo así la parte técnica de la red con el organigrama de la empresa.

Además, he montado un sistema para gestionar las especialidades de cada uno. En la vida real, no todos los informáticos dominan la nube o el análisis forense. Al organizar estas habilidades en la base de datos, el sistema puede cruzar datos al instante para decirme qué técnico Senior está cualificado y libre para arreglar una avería crítica en un cliente, optimizando a quién asigno sin tener que comprobarlo a mano.

2.3. Pilar 2: Infraestructura Física y Virtual

TechNova maneja equipos muy importantes y guarda información confidencial, así que el control del inventario tiene que ser muy estricto. He diseñado este bloque separando el hardware físico de las máquinas virtuales. Por un lado, guardo los equipos de red y los servidores físicos con sus direcciones IP y en qué rack exacto están, lo cual es clave para no perder tiempo si falla una pieza física. Por otro lado, vinculo las máquinas virtuales a los servidores físicos donde están alojadas, lo que me permite saber de un vistazo qué servicios de clientes se caerían si una máquina física pierde la conexión.

Dentro de este pilar, he metido un control muy fuerte sobre las copias de seguridad. Sé que una empresa tecnológica se hunde si no puede recuperarse de un desastre o un ataque de ransomware. Por eso, la base de datos incluye un registro diario que revisa el estado y el peso de los backups de cada máquina virtual, avisando automáticamente si alguna copia falla para asegurar que los sistemas están a salvo.

2.4. Pilar 3: Operaciones y Trazabilidad de Servicios

Este bloque agrupa el trabajo del día a día con los clientes y las tareas técnicas. He diseñado tablas para registrar los proyectos activos y controlar cuánta carga de trabajo hay. Al relacionar directamente a los técnicos con los proyectos y darles un límite de horas, evito que la gente se sature y que haya problemas porque alguien asuma más instalaciones de las que puede hacer.

A nivel de seguridad, he incluido un registro de alertas (logs) pensado para el Centro de Operaciones de Seguridad (SOC). Como futuro administrador, sé que es vital saber exactamente qué pasa en la red. Esta tabla guarda el nivel de gravedad, la hora exacta y la máquina virtual que está siendo atacada (por ejemplo, por un intento de fuerza bruta), permitiendo al equipo de seguridad actuar con prioridad y aislar los equipos antes de que el virus o la amenaza se extienda.

2.5. Pilar 4: Contabilidad y Facturación Escalable

Todo este montaje técnico se tiene que traducir al final en facturas correctas. He diseñado este módulo pensando en que sea fácil de mantener con el tiempo para no tener que hacer cambios pesados en el futuro. Por ejemplo, he sacado los impuestos (como el IVA) a una tabla aparte. De esta forma, si la ley cambia, solo tengo que cambiar un número en la base de datos para que se actualice en todas partes, evitando errores al hacer las facturas a mano.

Además, he separado la cabecera de la factura de las líneas con los detalles. Una consultora no cobra precios cerrados sin más, sino que factura horas de auditoría, licencias de software o equipos instalados. Esta separación en el diseño no solo hace que las cuentas estén súper claras para cualquier auditoría, sino que me permite sacar estadísticas exactas sobre qué servicios nos dan más dinero a final de año.



3. DISEÑO LÓGICO Y ESTRUCTURAL

Antes de ejecutar ninguna sentencia de código SQL, he diseñado el modelo lógico de la base de datos. Empezar a crear tablas sin planificar antes cómo se van a relacionar suele generar problemas y datos huérfanos. Por eso, he definido cada tabla con cuidado, eligiendo los tipos de datos exactos para no desperdiciar recursos del servidor, y dejando claras las reglas (cardinalidades) que unen todo el sistema.

3.1. ETAPA 1: GESTIÓN DE IDENTIDADES Y RECURSOS HUMANOS

- **Departamento:** Esta tabla es la base para organizar a la plantilla y, además, me sirve para conectar la base de datos con la red. He incluido un campo para guardar el ID de la **VLAN**, uniendo así la organización de la empresa con las subredes que planifiqué para TechNova.
 - **Cardinalidad:** Relación 1:N (Uno a Muchos) con *Empleado*. Un departamento agrupa a varios trabajadores, pero cada empleado solo pertenece a un departamento.
- **Empleado:** Aquí registro los datos de los trabajadores. Para los nombres y los cargos, he usado tipos de datos que se ajustan a la longitud real del texto para optimizar el espacio en disco. A nivel de seguridad, el correo electrónico tiene una restricción de unicidad para que el sistema bloquee intentos de duplicar correos y evitar brechas de acceso.
 - **Cardinalidad:** Tiene una clave foránea (FK) de *Departamento* (1:N). También tiene una relación N:M (Muchos a Muchos) con *Especialidad*, que he resuelto con una tabla intermedia.
- **Especialidad:** En vez de registrar manualmente en qué destaca cada técnico, he creado un catálogo centralizado. Al separar tecnologías como "Python" o "Cisco" en su propia tabla, evito datos repetidos o errores tipográficos, manteniendo la base de datos normalizada.
 - **Cardinalidad:** Relación N:M con *Empleado*. Una tecnología puede ser dominada por varios técnicos, y un técnico puede dominar varias tecnologías.
- **Especialidad_empleado (Tabla Puente):** Esta tabla sirve para resolver la relación "Muchos a Muchos" anterior. Une las claves de empleado y especialidad. Le he añadido un atributo "nivel" con una restricción (**CHECK**) para que solo acepte los valores "Junior", "Mid" o "Senior", asegurando que los datos entren estandarizados.
 - **Cardinalidad:** Recibe las relaciones 1:N desde *Empleado* y 1:N desde *Especialidad*.

3.2. ETAPA 2: CONTROL DE INFRAESTRUCTURA Y VIRTUALIZACIÓN

- **Equipo_fisico:** La he diseñado para llevar el inventario del hardware de red periférico (switches y firewalls). Guarda el modelo exacto y cuenta con una restricción de unicidad sobre la IP de gestión. Esto es vital para que, en caso de caída de la red, los técnicos localicen el equipo afectado sin confusiones.
 - **Cardinalidad:** Al ser hardware independiente de la virtualización, en esta fase opera como una entidad de inventario estático, sin claves foráneas directas.
- **Servidor_maestro:** Aquí registro los servidores físicos (hipervisores) que soportan la infraestructura. Guardo la memoria RAM y su ubicación en el rack, datos críticos para planificar el mantenimiento del Centro de Datos.
 - **Cardinalidad:** Relación 1:N con *Servidor_virtual*. Un servidor físico soporta varias máquinas virtuales, pero una máquina virtual corre en un servidor físico concreto.
- **Servidor_virtual:** Representa las máquinas donde se despliegan los servicios de los clientes. Guardo su sistema operativo y su IP interna.
 - **Cardinalidad:** Recibe la clave foránea de *Servidor_maestro* (1:N). Además, es el origen de dos relaciones 1:N hacia las tablas de copias de seguridad y avisos de seguridad.
- **Copia_seguridad:** Es fundamental para el plan de recuperación ante desastres (Disaster Recovery). Usa un formato **TIMESTAMP** para registrar el segundo exacto del backup, y una restricción **CHECK** para validar si el estado es 'Exitosa', 'Fallida' o 'En Proceso'.
 - **Cardinalidad:** Recibe la clave foránea de *Servidor_virtual* (1:N). Cada máquina virtual genera un historial con múltiples copias de seguridad.

3.3. ETAPA 3: OPERACIONES, PROYECTOS Y CIBERSEGURIDAD

- **Cliente:** Es el nodo central del negocio. Almacena los datos fiscales y de contacto. Le he aplicado una restricción de unicidad al CIF para evitar clientes duplicados y problemas de facturación.
 - **Cardinalidad:** Relación 1:N con *Proyecto* y 1:N con *Factura*. Un cliente puede generar varios proyectos y múltiples facturas a lo largo del tiempo.
- **Proyecto:** Controla la viabilidad económica de los servicios. Para los presupuestos, he utilizado el tipo **DECIMAL** en lugar de números con coma flotante, evitando así pérdidas o redondeos imprecisos en la contabilidad.

- **Cardinalidad:** Recibe la clave foránea de *Cliente* (1:N). Participa en una relación N:M con *Empleado* para la asignación de tareas.
- **Trabajo_empleado (Tabla Puente):** Resuelve la necesidad operativa de asignar técnicos a proyectos. Al crear esta tabla intermedia, he aprovechado para añadir el atributo "horas_asignadas". Así el departamento de recursos humanos puede auditar la carga laboral y evitar que un trabajador se sature.
 - **Cardinalidad:** Recibe las relaciones 1:N desde *Empleado* y 1:N desde *Proyecto*.
- **Aviso_seguridad:** Funciona como el registro centralizado de incidentes para el equipo SOC. Usa un tipo de texto extendido (**TEXT**) para permitir descripciones largas de ataques complejos, almacenando la fecha exacta y el nivel de gravedad.
 - **Cardinalidad:** Recibe la clave foránea de *Servidor_virtual* (1:N). Una máquina virtual puede sufrir múltiples intentos de ataque que quedan registrados en este historial.

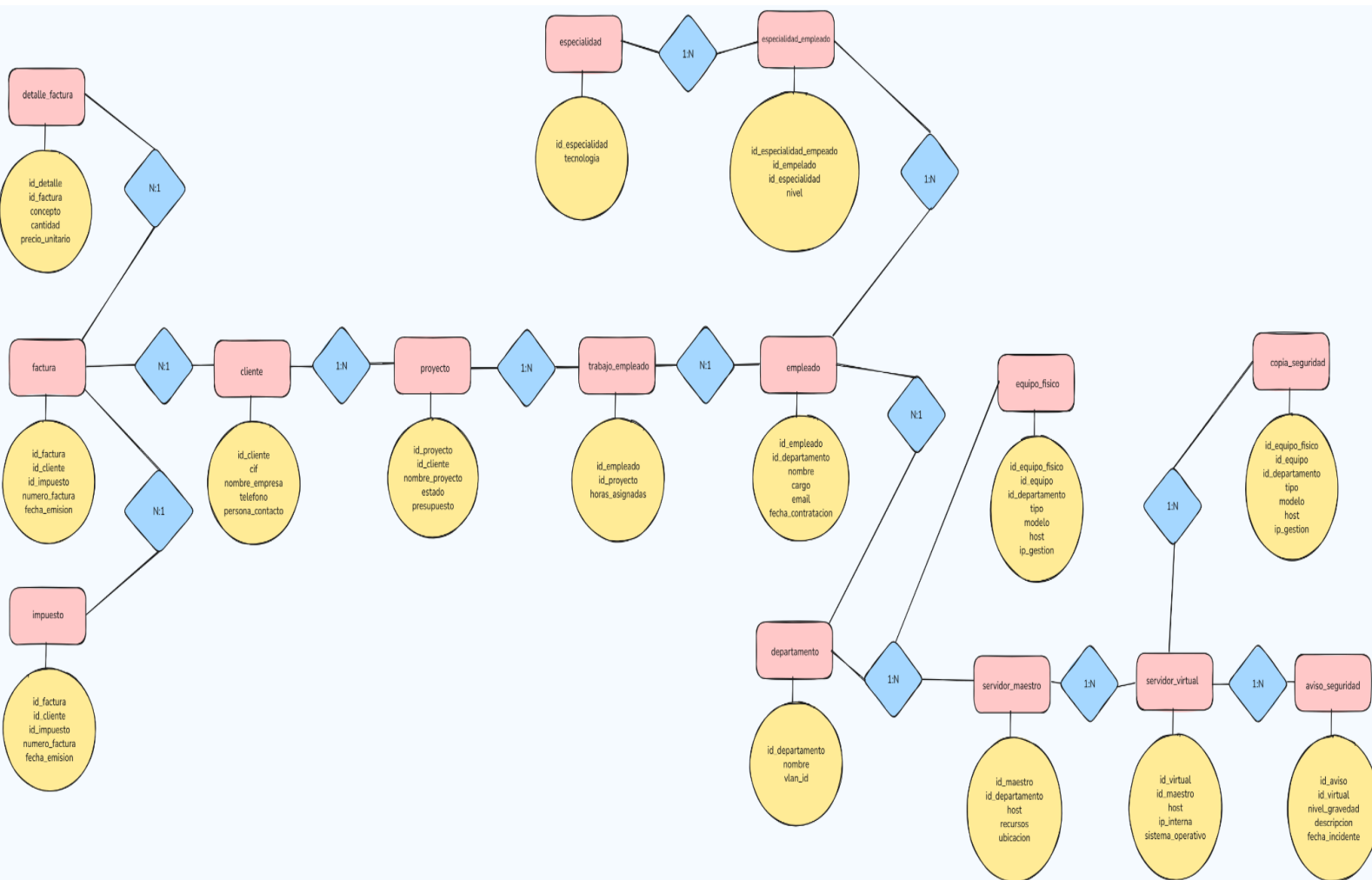
3.4. ETAPA 4: ESCALABILIDAD FINANCIERA Y FACTURACIÓN

- **Impuesto:** Aislar el tipo impositivo en una tabla propia es una decisión crítica de diseño. Si la legislación fiscal cambia, no tengo que actualizar miles de facturas; modifico un registro en esta tabla y el cálculo se aplica a todo el sistema automáticamente.
 - **Cardinalidad:** Relación 1:N con *Factura*, ya que un mismo tipo de IVA se aplica a infinidad de documentos fiscales.
- **Factura:** Implementa la cabecera del documento mercantil. Registra la fecha de emisión y genera un número de factura único.
 - **Cardinalidad:** Recibe las claves foráneas de *Cliente* (1:N) y de *Impuesto* (1:N). A su vez, mantiene una relación 1:N hacia *Detalle_factura*, porque una factura se compone de varias líneas de cobro.
- **Detalle_factura:** Desglosa los conceptos facturados (horas, licencias, hardware). Al separar esta tabla de la cabecera de la factura, el sistema permite procesar una factura de una sola línea o despliegues complejos sin redundancia de datos. Para los precios, vuelve a implementar el tipo **DECIMAL**.
 - **Cardinalidad:** Recibe la clave foránea de *Factura* (1:N). Varias líneas de detalle pertenecen siempre a un único documento de factura.

3.5. DIAGRAMA ENTIDAD-RELACIÓN (E-R)

Una vez definidas las reglas de negocio y las cardinalidades, el siguiente paso ha sido plasmar esta lógica en un esquema conceptual. El Diagrama Entidad-Relación (E-R) actúa como un plano visual de alto nivel de todo el sistema.

En esta fase inicial no intervienen sentencias SQL ni tipos de datos técnicos. El objetivo es representar gráficamente las entidades clave de la empresa y las acciones o verbos que las vinculan (por ejemplo, la relación donde un *Ciente* genera un *Proyecto*, o un *Empleado* pertenece a un *Departamento*).



3.6. EL PLANO LÓGICO: MODELO RELACIONAL Y NORMALIZACIÓN

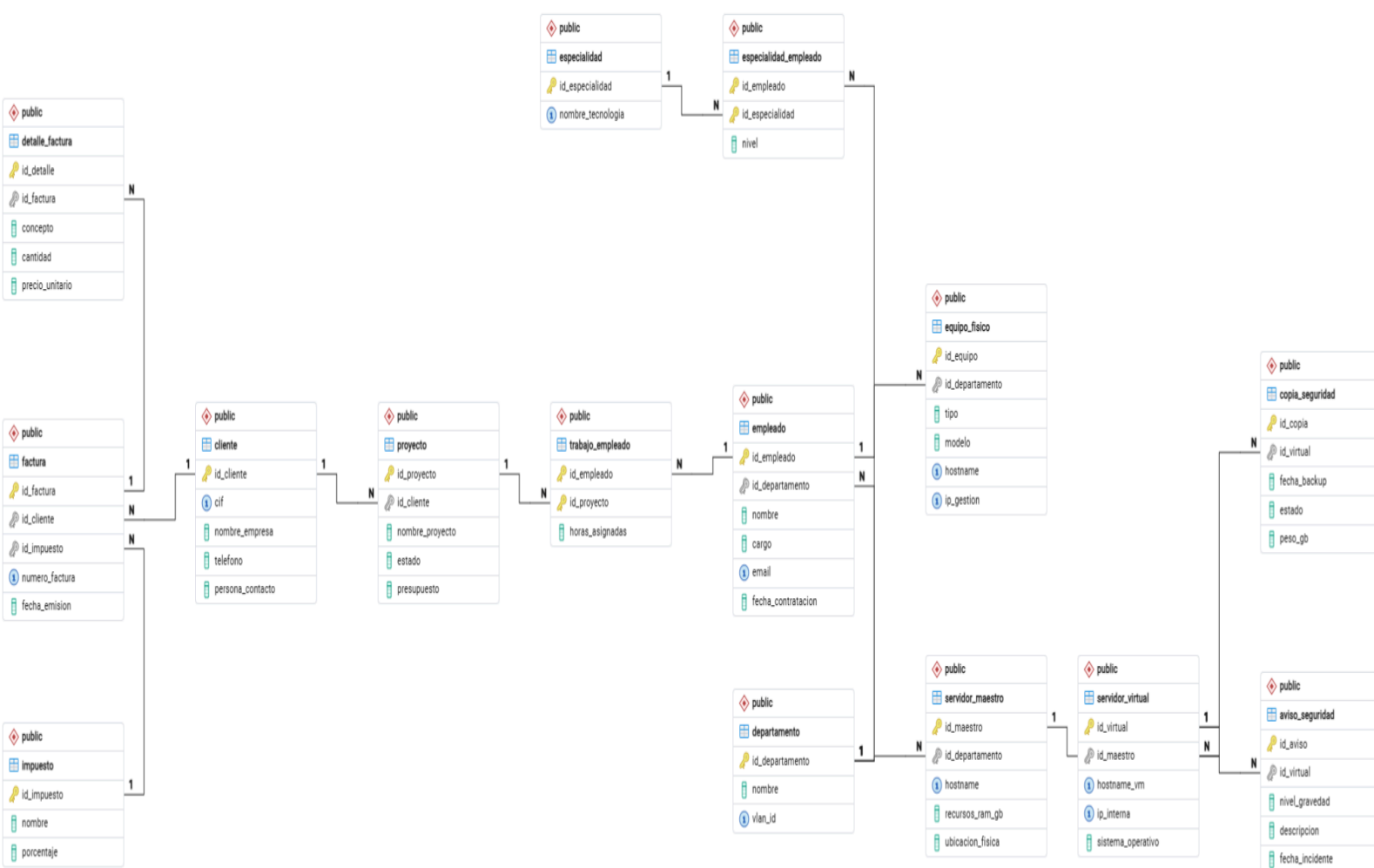
Una vez que el esquema conceptual está validado, he pasado toda esa lógica al Modelo Relacional. Aquí es donde defino exactamente las reglas que van a darle forma a la base de datos definitiva.

Para que el sistema sea eficiente, he aplicado las reglas de la Tercera Forma Normal (3FN) en todas las tablas, eliminando cualquier dato que estuviera repetido. Si evito que cosas

como el nombre de un departamento o un porcentaje de IVA se repitan en varias tablas a la vez, me ahorro problemas a la hora de actualizar la información y optimizo el espacio en el servidor.

En este paso también he dejado bien claras las Claves Primarias (PK), para asegurar que cada registro sea único, y las Claves Foráneas (FK), que son las que unen las tablas y mantienen la integridad referencial. Hacer este diseño previo es muy importante porque frena fallos graves desde el principio; por ejemplo, la base de datos no te va a dejar asignar un técnico a un servidor que no existe, ni borrar a un cliente si todavía tiene facturas por pagar.

Con este esquema ya normalizado y sin errores lógicos, la estructura queda completamente lista para empezar a programarla en SQL.



4. IMPLEMENTACIÓN ESTRUCTURAL (CÓDIGO DDL)

4.1. Traducción del Modelo Relacional a SQL (DDL)

Con el modelo lógico validado, he generado el script en SQL usando comandos **DDL** (Data Definition Language) para montar la estructura directamente en el servidor de base de datos. Al escribirlo, he elegido bien cada tipo de dato pensando en el rendimiento del sistema y en no malgastar espacio en disco.

Por ejemplo, he implementado el uso de **VARCHAR** para que los textos ocupen solo el espacio que realmente necesitan. También he aprovechado el tipo **SERIAL** para que el motor de **PostgreSQL** gestione los identificadores (IDs) de forma autoincremental, evitando problemas con claves duplicadas. En la parte financiera de la empresa (presupuestos y facturación), he evitado usar números de coma flotante (**FLOAT**) y he apostado por el tipo **DECIMAL**. Así me aseguro de que las cuentas sean exactas y no haya descuadres por problemas de redondeo en la contabilidad.

Una parte fundamental de este script ha sido configurar bien la integridad referencial para controlar qué pasa cuando se borran datos:

- **ON DELETE RESTRICT:** Lo he aplicado en las relaciones más críticas del negocio. Por ejemplo, el sistema bloquea el borrado de un cliente o departamento si todavía tienen proyectos o empleados activos, protegiendo así el histórico de datos de la empresa.
- **ON DELETE CASCADE:** Lo he usado en la capa de infraestructura virtual. De esta forma, si decido dar de baja una máquina virtual (**servidor_virtual**), el motor borra automáticamente todas sus copias de seguridad y alertas de seguridad asociadas. Con esto me ahorro tener que hacer limpiezas a mano y evito que queden datos "huérfanos" que acaben ralentizando el sistema.

4.2. Script de creación de la base de datos (Esqueleto de TechNova)

A continuación, presento el código DDL definitivo que he ejecutado para levantar la arquitectura de TechNova Solutions S.L. Está compuesto por las 15 tablas interconectadas, cumpliendo con todas las reglas de cardinalidad y normalización definidas en el diseño previo.

=====

-- FASE 0: LIMPIEZA DE ENTORNO (RE-RUNNABLE SCRIPT)

=====

-- He configurado este bloque inicial para poder ejecutar el script infinitas veces

-- desde cero. Utilizo CASCADE para garantizar que PostgreSQL elimine las tablas

-- respetando las dependencias jerárquicas y evitando bloqueos por claves foráneas.

```

DROP TABLE IF EXISTS detalle_factura CASCADE;
DROP TABLE IF EXISTS factura CASCADE;
DROP TABLE IF EXISTS impuesto CASCADE;
DROP TABLE IF EXISTS aviso_seguridad CASCADE;
DROP TABLE IF EXISTS trabajo_empleado CASCADE;
DROP TABLE IF EXISTS proyecto CASCADE;
DROP TABLE IF EXISTS cliente CASCADE;
DROP TABLE IF EXISTS copia_seguridad CASCADE;
DROP TABLE IF EXISTS servidor_virtual CASCADE;
DROP TABLE IF EXISTS servidor_maestro CASCADE;
DROP TABLE IF EXISTS equipo_fisico CASCADE;
DROP TABLE IF EXISTS especialidad_empleado CASCADE;
DROP TABLE IF EXISTS especialidad CASCADE;
DROP TABLE IF EXISTS empleado CASCADE;
DROP TABLE IF EXISTS departamento CASCADE;

```

```
=====
```

```
-- BLOQUE 1: RECURSOS HUMANOS Y ORGANIGRAMA
```

```
=====
```

Considero que el departamento es la base de la infraestructura de TechNova. He añadido una conexión directa con el área de Redes mediante el ID de VLAN.

```

CREATE TABLE departamento (
    id_departamento SERIAL PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    vlan_id INT NOT NULL UNIQUE -- Enlace fundamental con el diseño físico de
switches y routing
);

```

Para cuidar la integridad de la plantilla, aplico ON DELETE RESTRICT.

No permito que un departamento se elimine si todavía existen empleados trabajando en él.

```
CREATE TABLE empleado (  
    id_empleado SERIAL PRIMARY KEY,  
    id_departamento INT NOT NULL REFERENCES departamento(id_departamento)  
    ON DELETE RESTRICT,  
    nombre VARCHAR(100) NOT NULL,  
    cargo VARCHAR(100) NOT NULL,  
    email VARCHAR(100) UNIQUE NOT NULL,  
    fecha_contratacion DATE DEFAULT CURRENT_DATE  
);
```

-- Catálogo maestro de tecnologías para indexar el conocimiento de la empresa.

```
CREATE TABLE especialidad (  
    id_especialidad SERIAL PRIMARY KEY,  
    nombre_tecnologia VARCHAR(50) NOT NULL UNIQUE  
);
```

He decidido usar un CHECK para parametrizar el nivel técnico, limitando los rangos para evitar datos inconsistentes. Si un empleado es dado de baja, sus registros de conocimiento se eliminan en cascada.

```
CREATE TABLE especialidad_empleado (  
    id_empleado INT REFERENCES empleado(id_empleado) ON DELETE CASCADE,  
    id_especialidad INT REFERENCES especialidad(id_especialidad) ON DELETE  
    CASCADE,  
    nivel VARCHAR(20) CHECK (nivel IN ('Junior', 'Mid', 'Senior')),  
    PRIMARY KEY (id_empleado, id_especialidad)  
);
```

=====

-- BLOQUE 2: INFRAESTRUCTURA DE HARDWARE Y SEGURIDAD

=====

En el inventario de hardware he aplicado ON DELETE SET NULL. Si un departamento se disuelve, los equipos físicos (routers, PCs, racks) no desaparecen; pasan a estar sin asignar en el inventario general del administrador de sistemas.

```
CREATE TABLE equipo_fisico (  
    id_equipo SERIAL PRIMARY KEY,  
    id_departamento INT REFERENCES departamento(id_departamento) ON DELETE  
    SET NULL,  
    tipo VARCHAR(50) NOT NULL,  
    modelo VARCHAR(100) NOT NULL,  
    hostname VARCHAR(50) UNIQUE NOT NULL,  
    ip_gestion VARCHAR(15) UNIQUE -- IP del segmento de administración  
    (Out-of-Band)  
);
```

-- Representa el hardware dedicado (Bare-Metal) que actúa como hipervisor en el CPD.

```
CREATE TABLE servidor_maestro (  
    id_maestro SERIAL PRIMARY KEY,  
    id_departamento INT REFERENCES departamento(id_departamento) ON DELETE  
    SET NULL,  
    hostname VARCHAR(50) UNIQUE NOT NULL,  
    recursos_ram_gb INT NOT NULL,  
    ubicacion_fisica VARCHAR(100)  
);
```

-- Capa de virtualización. He aplicado ON DELETE CASCADE: si un servidor maestro físico

-- es desmantelado o destruido de la base de datos, sus instancias virtuales residentes

-- lógicamente dejan de existir en esa infraestructura.

```

CREATE TABLE servidor_virtual (
    id_virtual SERIAL PRIMARY KEY,
    id_maestro INT NOT NULL REFERENCES servidor_maestro(id_maestro) ON
DELETE CASCADE,
    hostname_vm VARCHAR(50) UNIQUE NOT NULL,
    ip_interna VARCHAR(15) UNIQUE NOT NULL,
    sistema_operativo VARCHAR(50) NOT NULL
);

```

Gestión del almacenamiento de respaldos para asegurar la continuidad del negocio.

He añadido un CHECK estricto para auditar los estados del proceso automatizado de backups.

```

CREATE TABLE copia_seguridad (
    id_copia SERIAL PRIMARY KEY,
    id_virtual INT NOT NULL REFERENCES servidor_virtual(id_virtual) ON DELETE
CASCADE,
    fecha_backup TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    estado VARCHAR(20) CHECK (estado IN ('Exitosa', 'Fallida', 'En Proceso')),
    peso_gb DECIMAL(8,2)
);

```

```

=====

```

```

-- BLOQUE 3: OPERACIONES, PROYECTOS Y MONITOREO

```

```

=====

```

```

-- Entidad externa que financia las operaciones de TechNova.

```

```

CREATE TABLE cliente (
    id_cliente SERIAL PRIMARY KEY,
    cif VARCHAR(20) UNIQUE NOT NULL,
    nombre_empresa VARCHAR(100) NOT NULL,
    telefono VARCHAR(20),

```



```

    persona_contacto VARCHAR(100)
);

-- Protejo el negocio con ON DELETE RESTRICT: bajo ninguna circunstancia se
puede
-- eliminar un cliente si tiene proyectos o contratos actualmente en ejecución.
CREATE TABLE proyecto (
    id_proyecto SERIAL PRIMARY KEY,
    id_cliente INT NOT NULL REFERENCES cliente(id_cliente) ON DELETE
    RESTRICT,
    nombre_proyecto VARCHAR(150) NOT NULL,
    estado VARCHAR(30) DEFAULT 'Activo',
    presupuesto DECIMAL(10,2) NOT NULL
);

```

Relación N:M para controlar el desglose de horas asignadas a la plantilla corporativa.

```

CREATE TABLE trabajo_employado (
    id_employado INT REFERENCES empleado(id_employado) ON DELETE CASCADE,
    id_proyecto INT REFERENCES proyecto(id_proyecto) ON DELETE CASCADE,
    horas_asignadas INT DEFAULT 0,
    PRIMARY KEY (id_employado, id_proyecto)
);

```

-- Módulo crítico de ciberseguridad. Diseñado para registrar eventos procedentes de
-- sistemas de monitoreo y auditoría de logs, clasificándolos mediante políticas de
impacto.

```

CREATE TABLE aviso_seguridad (
    id_aviso SERIAL PRIMARY KEY,
    id_virtual INT NOT NULL REFERENCES servidor_virtual(id_virtual) ON DELETE
    CASCADE,

```

```

        nivel_gravedad VARCHAR(20) CHECK (nivel_gravedad IN ('Bajo', 'Medio',
'Critico')),

        descripcion TEXT NOT NULL,

        fecha_incidente TIMESTAMP DEFAULT CURRENT_TIMESTAMP

);

```

```
=====
```

```
-- BLOQUE 4: FINANZAS Y CONTABILIDAD
```

```
=====
```

```
-- Tabla reguladora de tasas e impuestos aplicables a la facturación comercial.
```

```

CREATE TABLE impuesto (

    id_impuesto SERIAL PRIMARY KEY,

    nombre VARCHAR(50) NOT NULL,

    porcentaje DECIMAL(5,2) NOT NULL

);

```

Cabecera transaccional de cobros. Vinculada de forma restrictiva al clientey al impuesto para blindar la coherencia legal e impedir borrados accidentales.

```

CREATE TABLE factura (

    id_factura SERIAL PRIMARY KEY,

    id_cliente INT NOT NULL REFERENCES cliente(id_cliente) ON DELETE
RESTRICT,

    id_impuesto INT NOT NULL REFERENCES impuesto(id_impuesto) ON DELETE
RESTRICT,

    numero_factura VARCHAR(50) UNIQUE NOT NULL,

    fecha_emision DATE DEFAULT CURRENT_DATE

);

```

Detalle atómico de la factura. Aplico ON DELETE CASCADE: si una factura matriz es anulada o eliminada, todo su desglose económico asociado desaparece de forma conjunta.

```

CREATE TABLE detalle_factura (

    id_detalle SERIAL PRIMARY KEY,

    id_factura INT NOT NULL REFERENCES factura(id_factura) ON DELETE
    CASCADE,

    concepto VARCHAR(200) NOT NULL,

    cantidad INT DEFAULT 1,

    precio_unitario DECIMAL(10,2) NOT NULL

);

```

TechNova S.L/postgres@Local* X

TechNova S.L/postgres@Local

Query Query History

```

1
2  -- =====
3  -- FASE 0: LIMPIEZA DE ENTORNO (RE-RUNNABLE SCRIPT)
4  -- =====
5  -- He configurado este bloque inicial para poder ejecutar el script infinitas veces
6  -- desde cero. Utilizo CASCADE para garantizar que PostgreSQL elimine las tablas
7  -- respetando las dependencias jerárquicas y evitando bloqueos por claves foráneas.
8
9  DROP TABLE IF EXISTS detalle_factura CASCADE;
10 DROP TABLE IF EXISTS factura CASCADE;
11 DROP TABLE IF EXISTS impuesto CASCADE;
12 DROP TABLE IF EXISTS aviso_seguridad CASCADE;
13 DROP TABLE IF EXISTS trabajo_empleado CASCADE;
14 DROP TABLE IF EXISTS proyecto CASCADE;
15 DROP TABLE IF EXISTS cliente CASCADE;
16 DROP TABLE IF EXISTS copia_seguridad CASCADE;
17 DROP TABLE IF EXISTS servidor_virtual CASCADE;
18 DROP TABLE IF EXISTS servidor_maestro CASCADE;
19 DROP TABLE IF EXISTS equipo_fisico CASCADE;
20 DROP TABLE IF EXISTS especialidad_empleado CASCADE;
21 DROP TABLE IF EXISTS especialidad CASCADE;
22 DROP TABLE IF EXISTS empleado CASCADE;
23 DROP TABLE IF EXISTS departamento CASCADE;
24
25  -- =====
26  -- BLOQUE 1: RECURSOS HUMANOS Y ORGANIGRAMA
27  -- =====
28
29  -- Considero que el departamento es la base de la infraestructura de TechNova.
30  -- He añadido una conexión directa con el área de Redes mediante el ID de VLAN.
31  CREATE TABLE departamento (
32      id_departamento SERIAL PRIMARY KEY,
33      nombre VARCHAR(50) NOT NULL,
34      vlan_id INT NOT NULL UNIQUE -- Enlace fundamental con el diseño físico de switches y routing
35  );
36
37  -- Para cuidar la integridad de la plantilla, aplico ON DELETE RESTRICT.
38  -- No permito que un departamento se elimine si todavía existen empleados trabajando en él.
39  CREATE TABLE empleado (
40      id_empleado SERIAL PRIMARY KEY,
41      id_departamento INT NOT NULL REFERENCES departamento(id_departamento) ON DELETE RESTRICT,

```

5. INSERCIÓN Y VIDA DEL DATO (CÓDIGO DML)

5.1. Población del sistema: Vinculación entre asignaturas

Diseñar la estructura de las tablas es solo el primer paso; una base de datos vacía no sirve para comprobar si el modelo funciona en la realidad. La siguiente fase de mi proyecto ha sido llenar estas entidades usando el Lenguaje de Manipulación de Datos (DML), concretamente con la instrucción **INSERT**.

Los datos que he introducido no los he elegido al azar, sino que están totalmente conectados con los escenarios prácticos que he montado en los otros módulos. Por ejemplo, al dar de alta al equipo técnico de TechNova, los he asignado a departamentos que cuadran exactamente con las **VLANs (10, 20, 30, 40, 50)** que diseñé y justifiqué en la asignatura de Redes. De la misma forma, he registrado en el inventario los switches L2/L3 y los servidores físicos (maestros), calcando la topología de red y los hipervisores que configuré en Sistemas.

Esta carga de datos cubre todo el día a día operativo de la empresa: desde listar el catálogo de tecnologías, pasando por la asignación de horas de los técnicos a los proyectos, hasta simular incidentes críticos que registra el equipo de seguridad (SOC). Al meter toda esta información, el sistema "cobra vida". Esto me permite hacer pruebas reales para ver si las restricciones aguantan y comprobar que las consultas para sacar la facturación funcionan sin errores.

5.2. Script de inserción de datos operativos (DML)

A continuación, detallo el script SQL que he ejecutado para poblar las 15 tablas de la consultora. El código está escrito respetando estrictamente el orden de dependencias para no violar ninguna clave foránea durante la inserción.

SQL

```
=====
-- PROYECTO TECHNOVA: CÓDIGO DML (POBLACIÓN DE DATOS OPERATIVOS)
=====

-- 1. Estableciendo las Redes y Departamentos Lógicos
INSERT INTO departamento (nombre, vlan_id) VALUES
('Direccion', 10),
('Sistemas y Ciberseguridad', 20),
('Desarrollo e IA', 30),
('Soporte Administrativo y Comercial', 40),
('Red Perimetral Invitados', 50);

-- 2. Dando de alta a los expertos de TechNova
INSERT INTO empleado (id_departamento, nombre, cargo, email) VALUES
(1, 'Luis CEO', 'Chief Executive Officer', 'luis.ceo@technova.local'),
(1, 'Apoyo-Dir', 'Executive Assistant', 'apoyo.dir@technova.local'),
```

```
(2, 'Jaime', 'Responsable de Sistemas IT', 'jaime.sis@technova.local'),
(2, 'Alex Redes', 'Network & Routing Engineer', 'alex.redes@technova.local'),
(2, 'Carlos Bit', 'Systems Administrator', 'carlos.bit@technova.local'),
(2, 'Roberto Cyber', 'Cybersecurity SOC Analyst', 'roberto.cyber@technova.local'),
(2, 'Diego DevOps', 'DevOps & Automation Engineer', 'diego.devops@technova.local'),
(2, 'Ing-Rack', 'Data Center Technician', 'ing.rack@technova.local'),
(3, 'Lucía Lead', 'Desarrollo Lead', 'lucia.lead@technova.local'),
(3, 'Elena Cloud', 'Cloud Architect', 'elena.cloud@technova.local'),
(3, 'Víctor Backend', 'Backend Developer (Python)', 'victor.backend@technova.local'),
(3, 'María Frontend', 'Frontend Developer', 'maria.frontend@technova.local'),
(3, 'Hugo Data', 'Senior Data Analyst', 'hugo.data@technova.local'),
(3, 'Pau Neural', 'Data Scientist / IA', 'pau.neural@technova.local'),
(3, 'Ana Query', 'Database Administrator', 'ana.query@technova.local'),
(4, 'Sara Talento', 'HR & Talent Manager', 'sara.talento@technova.local'),
(4, 'Marta Ventas', 'Key Account Manager', 'marta.ventas@technova.local'),
(4, 'Carmen Legal', 'Legal & Compliance Officer', 'carmen.legal@technova.local'),
(4, 'Admin-01', 'Administrative Support', 'admin01@technova.local'),
(4, 'Admin-02', 'Administrative Support', 'admin02@technova.local');
```

-- 3. Catálogo de Tecnologías

```
INSERT INTO especialidad (nombre_tecnologia) VALUES
('Routing Cisco (CCNA)'), ('Security & Pentesting'), ('Python for Data Science'),
('AWS / Azure Cloud'), ('Virtualization (VMware)');
```

-- 4. Vinculando técnicos y especialidades (Relación N:M)

```
INSERT INTO especialidad_empleado (id_empleado, id_especialidad, nivel) VALUES
(4, 1, 'Senior'), (6, 2, 'Senior'), (10, 4, 'Senior'), (14, 3, 'Senior'), (15, 5, 'Mid');
```

-- 5. Inventariando la Infraestructura Física

```
INSERT INTO equipo_fisico (id_departamento, tipo, modelo, hostname, ip_gestion)
VALUES
(2, 'Gateway / Router L3', 'Cisco 2911', 'GW-TECHNOVA', '192.168.1.1'),
(2, 'Switch Core L3', 'Cisco Catalyst 3650', 'CORE-L3-3650', '192.168.1.254'),
(1, 'Switch Acceso L2', 'Cisco 2960', 'SW-ACC-DIR', '192.168.10.253'),
(2, 'Switch Acceso L2', 'Cisco 2960', 'SW-ACC-SIS', '192.168.20.253'),
(3, 'Switch Acceso L2', 'Cisco 2960', 'SW-ACC-DEV', '192.168.30.253'),
(4, 'Switch Acceso L2', 'Cisco 2960', 'SW-ACC-SOP', '192.168.40.253'),
(5, 'Access Point', 'AP-PT', 'AP-GUEST-WIFI', '192.168.50.253');
```

```
INSERT INTO servidor_maestro (id_departamento, hostname, recursos_ram_gb,
ubicacion_fisica) VALUES
(2, 'SRV-PROD-01', 256, 'Wiring Closet - Rack 1 - U10'),
(2, 'SRV-BACKUP-01', 128, 'Wiring Closet - Rack 1 - U12'),
(2, 'SRV-SVR-DHCP', 16, 'Wiring Closet - Rack 1 - U14'),
(2, 'SRV-SVR-DNS', 16, 'Wiring Closet - Rack 1 - U15');
```

-- 6. Despliegue de Máquinas Virtuales

```
INSERT INTO servidor_virtual (id_maestro, hostname_vm, ip_interna, sistema_operativo)
VALUES
```

```
(1, 'VM-AppServer-01', '192.168.20.10', 'Ubuntu 24.04 LTS'),
(1, 'VM-Database-01', '192.168.20.11', 'Windows Server 2022'),
(2, 'VM-Veeam-Vault', '192.168.20.100', 'Windows Server 2019');
```

-- 7. Registro de copias de seguridad (Disaster Recovery)

```
INSERT INTO copia_seguridad (id_virtual, estado, peso_gb) VALUES
```

```
(1, 'Exitosa', 150.75),
(2, 'Exitosa', 580.20),
(3, 'Exitosa', 45.00);
```

-- 8. Simulando incidentes registrados por el SOC

```
INSERT INTO aviso_seguridad (id_virtual, nivel_gravedad, descripcion, fecha_incidente)
VALUES
```

```
(1, 'Medio', 'Intentos de inicio de sesión no válidos en puerto SSH.', '2026-04-18 02:15:00'),
(2, 'Critico', 'Alerta de saturación de CPU (Posible ataque DDoS a Base de Datos).',
'2026-04-19 14:30:22');
```

-- 9. Altas de clientes y proyectos

```
INSERT INTO cliente (cif, nombre_empresa, telefono, persona_contacto) VALUES
```

```
('B87654321', 'Banco Santander Innovación', '912345678', 'Director Transformación'),
('A12345678', 'Hospital Clínico Central', '919876543', 'CISO Hospital');
```

```
INSERT INTO proyecto (id_cliente, nombre_proyecto, estado, presupuesto) VALUES
```

```
(1, 'Despliegue de IA para Detección de Fraudes (Copilot)', 'Activo', 85000.00),
(2, 'Auditoría Perimetral y Rediseño de VLANs Clínicas', 'Activo', 24500.00);
```

-- 10. Asignación operativa de horas al personal

```
INSERT INTO trabajo_empleado (id_empleado, id_proyecto, horas_asignadas) VALUES
```

```
(14, 1, 200), -- Pau Neural lidera la IA
(13, 1, 150), -- Hugo Data asiste con el dataset
(4, 2, 120), -- Alex Redes rediseña la red
(6, 2, 80); -- Roberto Cyber realiza la auditoría
```

-- 11. Configuración de Impuestos y emisión de Factura cabecera

```
INSERT INTO impuesto (nombre, porcentaje) VALUES ('IVA General (España)', 21.00);
```

```
INSERT INTO factura (id_cliente, id_impuesto, numero_factura, fecha_emision) VALUES
```

```
(2, 1, 'FAC-TECH-2026-001', '2026-04-20');
```

-- 12. Desglose transaccional de los conceptos cobrados

```
INSERT INTO detalle_factura (id_factura, concepto, cantidad, precio_unitario) VALUES
```

```
(1, 'Horas Consultoría Arquitectura Redes (Senior)', 120, 80.00),
(1, 'Horas Auditoría Pentesting (SOC Analyst)', 80, 95.00),
(1, 'Licencia Firewall y Gestión Anual', 1, 3500.00);
```

6. EXPLOTACIÓN DE DATOS: GENERACIÓN DE INTELIGENCIA DE NEGOCIO

6.1. La importancia de extraer métricas para la empresa

Una base de datos bien diseñada no sirve de mucho si luego no podemos sacar la información rápido para tomar decisiones. En esta parte del proyecto, he preparado consultas avanzadas usando la instrucción **SELECT** y cruces de tablas (**JOIN**).

La idea no es hacer listas simples por rellenar, sino poner a prueba el motor de la base de datos con situaciones reales del día a día de TechNova. Con esto, demuestro que el sistema puede procesar información de varias tablas a la vez en milisegundos y sacar reportes exactos sobre cómo está rindiendo la empresa y qué recursos estamos consumiendo.

6.2. Auditoría de Recursos Humanos (Control de carga laboral)

El primer problema a resolver era evitar que los técnicos se saturen de trabajo. El departamento de Recursos Humanos necesita saber al instante en qué proyecto está metido cada empleado, a qué departamento pertenece y cuántas horas tiene asignadas, para no organizar las cosas a ciegas.

Para automatizar esto, he diseñado una consulta que cruza cuatro tablas distintas (*empleado*, *departamento*, *proyecto* y la tabla intermedia *trabajo_empleado*) uniéndolas de forma segura mediante **JOIN**. Además, al ordenarlo todo con un **ORDER BY DESC** sobre las horas asignadas, el sistema me saca una lista priorizada: pone arriba del todo a los técnicos que tienen más carga de trabajo. Así, la empresa puede ver quién está al límite y repartir mejor las tareas antes de que haya problemas.

SQL

SELECT

 e.nombre AS Empleado_Especialista,
 d.nombre AS Departamento,
 p.nombre_proyecto AS Proyecto_Activo,
 te.horas_asignadas

FROM empleado e

JOIN departamento d ON e.id_departamento = d.id_departamento

JOIN trabajo_empleado te ON e.id_empleado = te.id_empleado

JOIN proyecto p ON te.id_proyecto = p.id_proyecto

ORDER BY te.horas_asignadas DESC;

```
1 SELECT
2     e.nombre AS Empleado_Especialista,
3     d.nombre AS Departamento,
4     p.nombre_proyecto AS Proyecto_Activo,
5     te.horas_asignadas
6 FROM empleado e
7 JOIN departamento d ON e.id_departamento = d.id_departamento
8 JOIN trabajo_empleado te ON e.id_empleado = te.id_empleado
9 JOIN proyecto p ON te.id_proyecto = p.id_proyecto
10 ORDER BY te.horas_asignadas DESC;
```

Data Output Messages Notifications

	empleado_especialista character varying (100)	departamento character varying (50)	proyecto_activo character varying (150)	horas_asignadas integer
1	Pau Neural	Desarrollo e IA	Despliegue de IA para Detección de Fraudes (Copi...	200
2	Hugo Data	Desarrollo e IA	Despliegue de IA para Detección de Fraudes (Copi...	150
3	Alex Redes	Sistemas y Cibersegurid...	Auditoría Perimetral y Rediseño de VLANs Clínicas	120
4	Roberto Cyber	Sistemas y Cibersegurid...	Auditoría Perimetral y Rediseño de VLANs Clínicas	80

6.3. Auditoría de Ciberseguridad (Trazabilidad y mitigación de incidentes)

El siguiente paso era cubrir las necesidades del Centro de Operaciones de Seguridad (SOC). Si hay un ataque, el tiempo de respuesta es clave; el equipo de seguridad no puede perder el tiempo revisando cientos de registros a mano para encontrar el problema. La base de datos tiene que darles la información exacta al instante para proteger la red.

Para solucionarlo, he preparado una consulta pensada para frenar amenazas rápido. Usando la cláusula **WHERE**, filtro todo el "ruido" y saco solo los avisos con gravedad 'Crítico'. Al mismo tiempo, haciendo un **JOIN** con la tabla *servidor_virtual*, consigo la dirección IP interna exacta de la máquina afectada. Al ordenar los resultados por fecha (**ORDER BY DESC**), el equipo de seguridad ve primero los ataques más recientes, lo que les permite aislar el servidor de la red antes de que el problema se extienda.

```
SQL
SELECT
    av.fecha_incidente,
    sv.hostname_vm AS Servidor_Atacado,
    sv.ip_interna AS Direccion_IP_Bloquear,
```



```

    av.descripcion AS Motivo_Critico
FROM aviso_seguridad av
JOIN servidor_virtual sv ON av.id_virtual = sv.id_virtual
WHERE av.nivel_gravedad = 'Critico'
ORDER BY av.fecha_incidente DESC;

```

```

1  SELECT
2      av.fecha_incidente,
3      sv.hostname_vm AS Servidor_Atacado,
4      sv.ip_interna AS Direccion_IP_Bloquear,
5      av.descripcion AS Motivo_Critico
6  FROM aviso_seguridad av
7  JOIN servidor_virtual sv ON av.id_virtual = sv.id_virtual
8  WHERE av.nivel_gravedad = 'Critico'
9  ORDER BY av.fecha_incidente DESC;
10

```

Data Output Messages Notifications

	fecha_incidente timestamp without time zone	servidor_atacado character varying (50)	direccion_ip_bloquear character varying (15)	motivo_critico text
1	2026-04-19 14:30:22	VM-Database-01	192.168.20.11	Alerta de saturación de CPU (Posible ataque DDoS a Base de Dat...

6.4. Auditoría Contable (Automatización de facturación)

El último reto era para el departamento financiero. Hacer facturas exige precisión; depender de cálculos a mano o de pasar datos a un Excel para sumar los cobros y aplicar el IVA genera un margen de error que no nos podemos permitir. Es la propia base de datos la que tiene que encargarse de calcular todo de forma automática.

Para automatizar este proceso, he diseñado una consulta que saca el resumen exacto de una factura concreta (en este caso, la del Hospital Clínico Central). Cruzando las tablas de *factura*, *cliente*, *impuesto* y *detalle_factura*, el sistema junta la cabecera del documento con todas las líneas que se van a cobrar.

Lo más potente de esta consulta es el uso de la función de suma (**SUM**) combinada con la cláusula **GROUP BY**. Por dentro, el servidor multiplica las cantidades por los precios unitarios para sacar la base imponible exacta. Inmediatamente después, cruza ese subtotal con el porcentaje de IVA de su propia tabla y aplica la fórmula matemática para dar el "Total a Pagar" definitivo. El resultado es un cálculo al instante y sin fallos, cuadrando las cuentas de TechNova a la perfección.

SQL

```

SELECT
    f.numero_factura,
    c.nombre_empresa AS Nombre_Cliente,
    SUM(df.cantidad * df.precio_unitario) AS Base_Imponible_Euros,
    i.porcentaje AS IVA_Aplicado,
    (SUM(df.cantidad * df.precio_unitario) * (1 + (i.porcentaje / 100))) AS
Total_A_Pagar_Con_IVA
FROM factura f
JOIN cliente c ON f.id_cliente = c.id_cliente
JOIN impuesto i ON f.id_impuesto = i.id_impuesto
JOIN detalle_factura df ON f.id_factura = df.id_factura
WHERE f.numero_factura = 'FAC-TECH-2026-001'
GROUP BY f.numero_factura, c.nombre_empresa, i.porcentaje;

```

Query Query History

```

1  SELECT
2      f.numero_factura,
3      c.nombre_empresa AS Nombre_Cliente,
4      SUM(df.cantidad * df.precio_unitario) AS Base_Imponible_Euros,
5      i.porcentaje AS IVA_Aplicado,
6      (SUM(df.cantidad * df.precio_unitario) * (1 + (i.porcentaje / 100))) AS Total_A_Pagar_Con_IVA
7  FROM factura f
8  JOIN cliente c ON f.id_cliente = c.id_cliente
9  JOIN impuesto i ON f.id_impuesto = i.id_impuesto
10 JOIN detalle_factura df ON f.id_factura = df.id_factura
11 WHERE f.numero_factura = 'FAC-TECH-2026-001'
12 GROUP BY f.numero_factura, c.nombre_empresa, i.porcentaje;
13

```

Data Output Messages Notifications

	numero_factura character varying (50)	nombre_cliente character varying (100)	base_imponible_euros numeric	iva_aplicado numeric (5,2)	total_a_pagar_con_iva numeric
1	FAC-TECH-2026-001	Hospital Clínico Central	20700.00	21.00	25047.000000000000000000000000

7. ADMINISTRACIÓN, MANTENIMIENTO Y SEGURIDAD

Construir y llenar la base de datos es solo el principio. En un entorno real, la información está expuesta a fallos internos y amenazas externas. Como administrador, mi prioridad es garantizar que el sistema de TechNova sea seguro y resistente. Para blindar la infraestructura, he configurado las siguientes políticas de seguridad y control de acceso.

7.1. Control de Acceso y Gestión de Privilegios (DCL)

Dar acceso total a todo el mundo es un peligro para la seguridad. Para evitarlo, he usado el Lenguaje de Control de Datos (DCL) diseñando una política basada en el **principio de menor privilegio**. He creado perfiles distintos para asegurarme de que cada departamento use únicamente los datos que necesita para trabajar:

- **Superusuario (DBA):** Es el perfil de administración con control total (**GRANT ALL PRIVILEGES**). Su uso se reserva exclusivamente para tareas de mantenimiento y cambios en la estructura.
- **Recursos Humanos (Sin permiso de borrado):** Tienen permisos específicos (**GRANT SELECT, INSERT, UPDATE**) sobre las tablas de los trabajadores. He quitado a propósito el comando **DELETE**; si un empleado se va de la empresa, su ficha se desactiva pero nunca se borra. Así evitamos perder el histórico de las horas que ha facturado y mantenemos la integridad de los datos.
- **Contabilidad (Aislamiento de red):** Tienen control sobre los clientes y las facturas, pero les he quitado el acceso (**REVOKE**) a toda la parte de infraestructura informática. Un contable no necesita saber las IPs de los servidores ni ver las alertas del SOC. Separar estas funciones evita modificaciones por error y mejora la seguridad interna.

A continuación, presento el script DCL que he ejecutado para aplicar este blindaje en el sistema:

```
=====
-- PROYECTO TECHNOVA: SCRIPT DCL (CREACIÓN DE ROLES Y PRIVILEGIOS)
=====

-- Conceder acceso base al esquema público
GRANT USAGE ON SCHEMA public TO public;

-- =====
-- PERFIL 1: DEPARTAMENTO DE RECURSOS HUMANOS
-- =====
-- Creación del usuario con contraseña segura
CREATE ROLE rol_rrhh WITH LOGIN PASSWORD 'TechNova_HR_2026*';

-- Permisos estrictos (SELECT, INSERT, UPDATE). Prohibido el DELETE.
GRANT SELECT, INSERT, UPDATE ON empleado, departamento, especialidad,
especialidad_empleado, trabajo_empleado TO rol_rrhh;
```

```
-- Permiso para que los IDs autonuméricos (SERIAL) funcionen al dar de alta
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO rol_rrhh;

-- =====
-- PERFIL 2: DEPARTAMENTO DE CONTABILIDAD
-- =====
-- Creación del usuario con contraseña segura
CREATE ROLE rol_contabilidad WITH LOGIN PASSWORD 'TechNova_Finance_2026*';

-- Permisos totales sobre el flujo de dinero y clientes
GRANT SELECT, INSERT, UPDATE, DELETE ON cliente, factura, detalle_factura, impuesto
TO rol_contabilidad;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO rol_contabilidad;

-- Bloqueo explícito de seguridad (Auditoría Cero Trust)
-- Se prohíbe rotundamente al área contable ver la infraestructura IT o de Seguridad
REVOKE ALL PRIVILEGES ON aviso_seguridad, servidor_virtual, servidor_maestro,
equipo_fisico, copia_seguridad FROM rol_contabilidad;
```

7.2. Política de Copias de Seguridad y Disaster Recovery (Backups)

La pérdida de la base de datos supondría la parada total de TechNova. Por ello, el diseño del sistema termina con una política de Continuidad de Negocio basada en la estrategia estándar de seguridad 3-2-1 (tres copias de seguridad, en dos tipos de soporte distintos y una de ellas fuera de la empresa).

Para asegurar que los datos estén disponibles sin tener que hacerlo a mano, he automatizado el proceso con las siguientes pautas técnicas:

- **Automatización nocturna (cron):** A nivel del sistema operativo Linux, he programado una tarea en el planificador que se ejecuta todos los días a las 03:00 a.m. Esta ventana de mantenimiento asegura que el proceso no consuma recursos del servidor ni afecte al rendimiento durante el horario de trabajo.
- **Volcado lógico (pg_dump):** En lugar de copiar los archivos del disco duro directamente, utilizo esta herramienta para extraer un archivo con todas las sentencias SQL (tanto la estructura como los datos) necesarias para reconstruir la base de datos desde cero. Al ser texto plano, evito arrastrar fallos de los bloques del disco físico.
- **Compresión y almacenamiento externo:** El script comprime el archivo resultante en formato **.tar.gz** para que ocupe menos espacio y lo envía de forma segura a un servidor externo (fuera de la oficina).
- **Plan de Disaster Recovery:** Si el servidor local sufre un problema grave o un ataque de ransomware, el protocolo a seguir es levantar un nuevo servidor limpio en la nube e importar este archivo comprimido en el motor de base de datos, recuperando toda la actividad de TechNova en pocos minutos.

A continuación, presento el script en Bash que he programado para organizar todo este proceso en nuestros servidores:

```
#!/bin/bash
#
=====
# SCRIPT AUTOMATIZADO DE COPIAS DE SEGURIDAD - TECHNOVA SOLUTIONS S.L.
# Ejecución programada en CRON: 0 3 * * * /opt/scripts/backup_technova.sh
#
=====

# 1. Definir variables (Añadiendo timestamp para no sobrescribir históricos)
FECHA=$(date +"%Y_%m_%d_%H%M")
ARCHIVO_SQL="/backups/local/TechNova_DB_Dump_$FECHA.sql"
ARCHIVO_ZIP="/backups/local/TechNova_DB_Dump_$FECHA.tar.gz"

echo "=====
echo "[+] Iniciando Backup Lógico de la BBDD TechNova..."

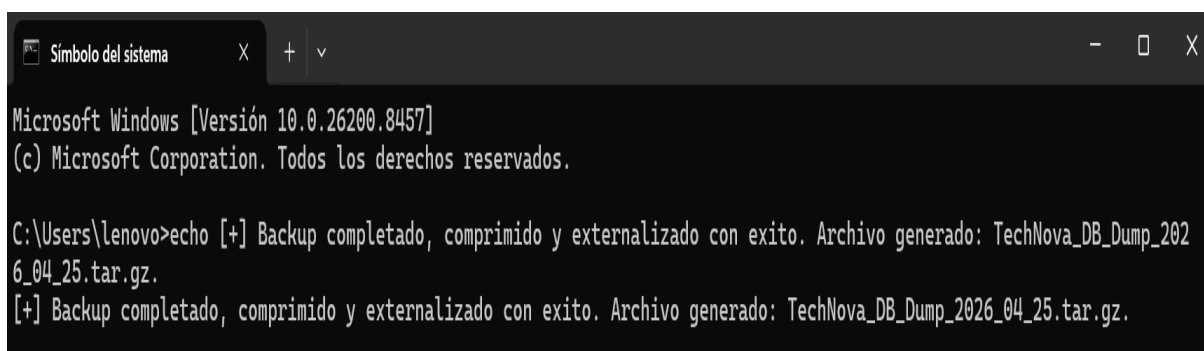
# 2. Ejecución del volcado lógico de la base de datos (pg_dump)
pg_dump -U postgres -d technova_db > $ARCHIVO_SQL

# 3. Compresión tar.gz para minimizar el peso del archivo de texto
tar -czvf $ARCHIVO_ZIP $ARCHIVO_SQL

# 4. Eliminación del archivo SQL plano por razones de seguridad espacial
rm $ARCHIVO_SQL

# 5. Simulación de externalización a servidor seguro en la nube mediante SCP (Regla 3-2-1)
# scp $ARCHIVO_ZIP admin_cloud@192.168.100.50:/vault_seguro_offsite/

echo "[+] Backup completado, comprimido y externalizado con éxito."
echo "[+] Archivo generado: $ARCHIVO_ZIP"
echo "=====
```



```
Microsoft Windows [Versión 10.0.26200.8457]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\lenovo>echo [+] Backup completado, comprimido y externalizado con éxito. Archivo generado: TechNova_DB_Dump_2026_04_25.tar.gz.
[+] Backup completado, comprimido y externalizado con éxito. Archivo generado: TechNova_DB_Dump_2026_04_25.tar.gz.
```

8. CONCLUSIONES FINALES Y ESCALABILIDAD DEL MODELO

8.1. ¿Por qué esta base de datos está preparada para el futuro?

A lo largo de este informe he querido demostrar que el diseño de una base de datos va mucho más allá de introducir comandos en una terminal. Ha requerido un análisis profundo, lógica de negocio y visión de futuro. El modelo final está preparado para crecer sin romperse gracias a tres factores clave:

- **La Normalización:** Al aplicar estrictamente la Tercera Forma Normal (3FN), hemos evitado los datos repetidos. Si el nombre de un cliente cambia o si el Gobierno modifica la ley de impuestos, solo tenemos que editar una fila en una tabla catálogo. El resto del sistema se actualiza de forma automática y limpia.
- **Cimientos independientes:** He separado a propósito los conceptos para que la empresa no tenga cuellos de botella informáticos. Un claro ejemplo es haber dividido la cabecera de la *Factura* de su *Detalle*. Gracias a esto, el sistema permite hacer facturas de una sola línea (por un servicio rápido) o facturas enormes con miles de elementos (por instalar infraestructura en múltiples sedes), sin que la base de datos sufra la más mínima ralentización.
- **Visión integrada entre asignaturas:** Este no es un proyecto aislado de bases de datos, sino que he fusionado esta capa lógica con el resto de módulos de ASIR. La tabla de *Departamento* se conecta con los parámetros de las **VLANs** de la asignatura de Redes, y la tabla de *Servidores Virtuales* asimila los conceptos de hipervisores que hemos estudiado en Sistemas Operativos.

8.2. Reflexión personal

Desarrollar este núcleo de información para TechNova S.L. ha sido el ejercicio más integrador de mi año académico. He comprobado que la infraestructura tecnológica, si no sirve para organizar, proteger o facilitar la actividad del negocio, pierde gran parte de su valor.

Saber cruzar tablas con claves foráneas, proteger los registros frente a borrados accidentales y diseñar modelos preparados para extraer información a través de consultas SQL son las competencias clave que distinguen a un buen administrador de sistemas. Terminé este módulo comprendiendo que una base de datos bien diseñada es, en última instancia, el cimiento técnico que sostiene la confianza de los clientes y los sistemas de cualquier empresa tecnológica.